



LINT REPORT

Fecha: 08/05/2026

G0-65

Contenido

1. Resumen ejecutivo 2

2. Historial de revisiones 2

3. Introducción 2

4. Análisis de “Bad Smells” Justificados..... 3

5. Conclusiones..... 3

6. Bibliografía 4

1. Resumen ejecutivo

El presente documento detalla los resultados del análisis estático de código realizado sobre el proyecto de Java Acme-Starters utilizando el plugin SonarLint en el entorno de desarrollo Eclipse. El objetivo principal de este análisis es asegurar la calidad del código, mantenibilidad y adherencia a los estándares de programación. Durante el proceso, se han subsanado la mayoría de los "bad smells" y vulnerabilidades detectadas por la herramienta. Sin embargo, este informe recopila y justifica técnica y lógicamente aquellas advertencias emitidas por SonarLint que se han considerado inofensivas (falsos positivos) para el contexto específico de este proyecto, demostrando así un uso crítico de las herramientas de calidad de software. Se han omitido deliberadamente las advertencias relativas al *Acme Framework*, de acuerdo con las especificaciones del proyecto.

2. Historial de revisiones

Revisión	Fecha	Descripción
1.0	08/05/2026	Creación del documento y primer análisis
1.1	08/05/2026	Corrección de "bad smells"

3. Introducción

La calidad del software es un pilar fundamental en el desarrollo de aplicaciones robustas. Para garantizar dicha calidad en el proyecto Acme-Starters se ha integrado la herramienta SonarLint en el flujo de trabajo de Eclipse. Esta herramienta permite la detección temprana de patrones de código problemáticos, conocidos como "bad smells", que podrían derivar en errores, problemas de rendimiento o dificultades de mantenimiento a futuro.

El equipo de desarrollo ha revisado exhaustivamente cada una de las advertencias generadas. Se ha aplicado el principio de refactorización constante, corrigiendo el código subyacente para eliminar la mayoría de los reportes. No obstante, las herramientas de análisis estático no comprenden el contexto completo del

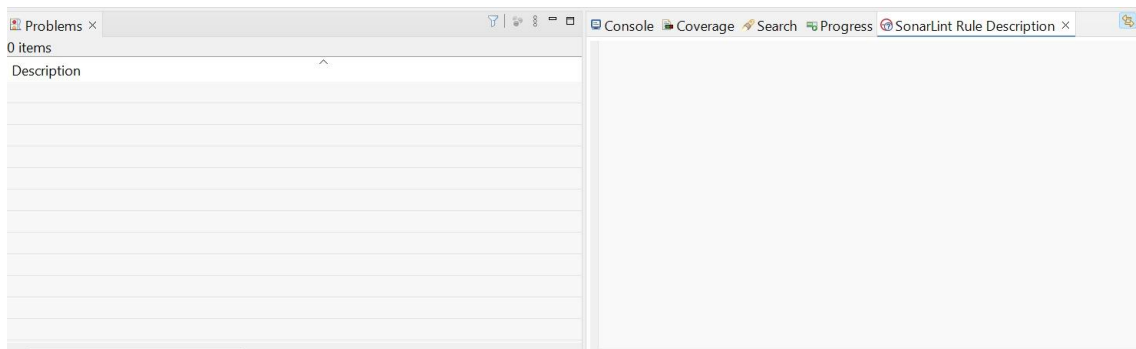
dominio del negocio o ciertas decisiones arquitectónicas puntuales. Por ello, existen advertencias que, tras ser consultadas y evaluadas, se han marcado como inofensivas.

El resto de este documento se estructura de la siguiente manera: la Sección 4 expone un listado detallado de los "bad smells" detectados por SonarLint que no han sido modificados, acompañados de su correspondiente justificación técnica. La Sección 5 resume las conclusiones obtenidas tras este proceso de auditoría de código. Finalmente, la Sección 6 contiene las referencias bibliográficas utilizadas, si las hubiere.

4. Análisis de “Bad Smells” Justificados

Tras la auditoría final realizada con SonarLint sobre el código fuente del proyecto, no se reporta ningún "bad smell", vulnerabilidad ni error de estilo en las clases desarrolladas (el entorno de desarrollo indica "0 items" en la vista de problemas).

Todos los avisos detectados en fases iniciales y durante el desarrollo fueron evaluados rigurosamente y corregidos directamente en el código fuente para cumplir con los máximos estándares de calidad exigidos, aplicando los principios de código limpio (clean code). Puesto que no queda ninguna advertencia activa que requiera justificación por ser un falso positivo (y habiendo omitido deliberadamente cualquier advertencia proveniente del Acme Framework, de acuerdo con las especificaciones de la práctica), este apartado queda intencionadamente sin entradas.



5. Conclusiones

El uso de SonarLint como analizador estático integrado en Eclipse ha resultado ser una práctica altamente beneficiosa y determinante para el éxito del proyecto. Durante el ciclo de desarrollo, la herramienta permitió establecer un estándar de calidad estricto, previniendo posibles errores de ejecución y mejorando la legibilidad general del código en Java. El enfoque de trabajo adoptado ha sido el de

refactorización constante y corrección preventiva, priorizando siempre solucionar el código subyacente antes que acumular deuda técnica o justificar excepciones.

Como resultado directo de esta disciplina, la auditoría final refleja un código completamente limpio, logrando el objetivo de cero "bad smells" reportados en las clases de la aplicación (excluyendo el marco de trabajo Acme Framework, conforme a las directrices del proyecto). Este resultado garantiza que la arquitectura y la implementación presentan un alto estándar de limpieza, robustez y mantenibilidad. En conclusión, el proceso de validación y limpieza exigido por el linting ha enriquecido el criterio técnico aplicado, asegurando que el software entregado es fiable y está preparado de manera óptima para futuras fases de mantenimiento o escalabilidad.

6. Bibliografía

Intencionalmente en blanco